

Hashing Patterns for DSA

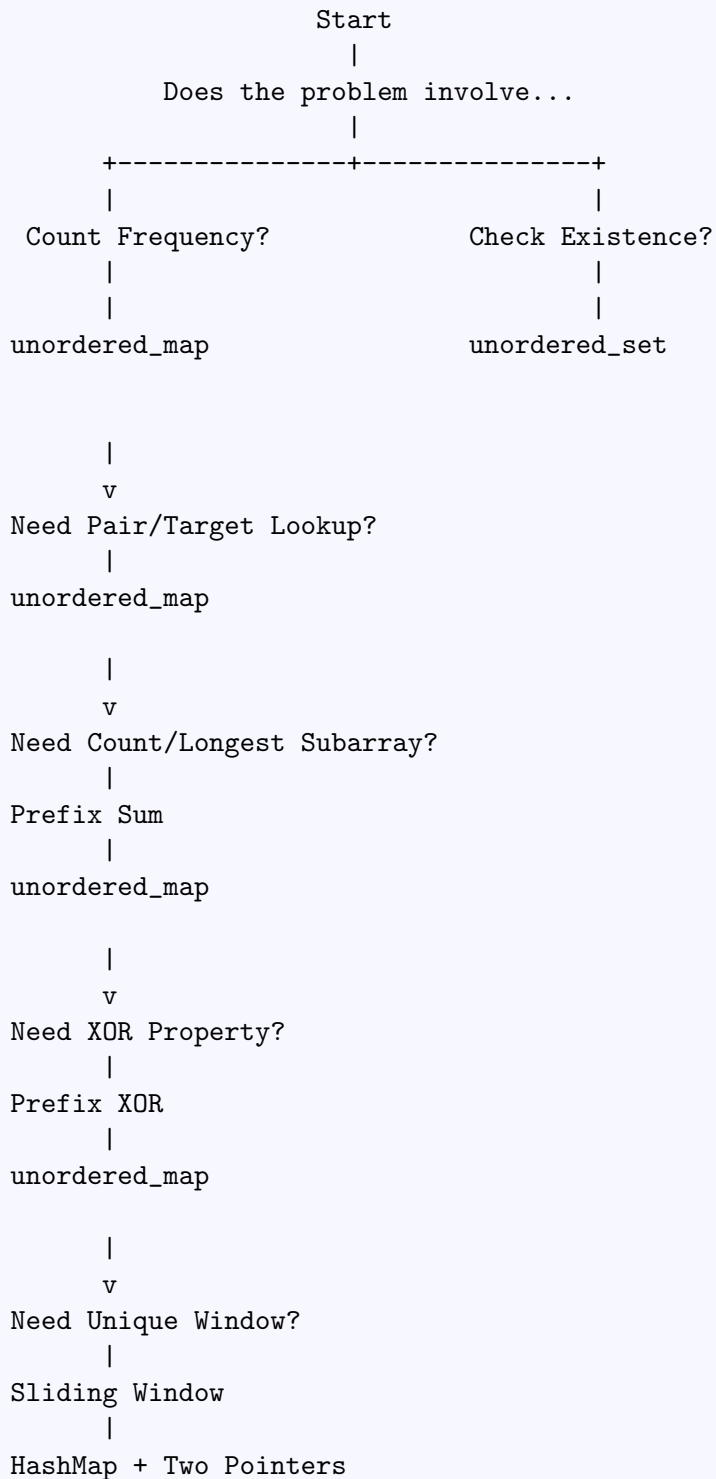
Contents

1	How to Identify a Hashing Problem Intuitively	2
2	STL	5
2.1	unordered_map	5
2.2	unordered_set	5
3	Complexities	6
4	Core Patterns	6
4.1	Frequency Counting	6
4.2	Lookup Pattern	6
4.3	Prefix Sum + Hash Map	6
4.4	Prefix XOR	7
4.5	Sliding Window + Hashing	7
5	Recognition Table	7
6	Common Mistakes	7
7	Important Interview Problems	7

1 How to Identify a Hashing Problem Intuitively

One of the biggest challenges in coding interviews is not writing the code—it is identifying *which technique* to use. Before thinking about algorithms, ask yourself a few simple questions about the problem statement.

Hashing Decision Tree



Understanding Each Decision

Rather than memorizing algorithms, think about the **information** you need to remember while processing the array.

1. Do I need to count how many times an element appears?

If the problem asks for frequencies, majority elements, duplicates, or counting occurrences, then we need a mapping from an element to its count.

Element $\rightarrow$ Frequency

Use:

```
unordered_map<Key, int>
```

Examples:

- Majority Element
- Top K Frequent Elements
- Group Anagrams
- Frequency Count

2. Do I only care whether an element exists?

Sometimes frequency is unnecessary. We only need to know whether an element has been seen before.

Element $\rightarrow$ Present / Not Present

Use:

```
unordered_set<Key>
```

Examples:

- Contains Duplicate
- Longest Consecutive Sequence
- Intersection of Arrays

3. Do I need to quickly find a complement?

If every element needs another element to satisfy a condition (usually a target sum), then store previously visited elements.

Value $\rightarrow$ Index

Use:

```
unordered_map<Value, Index>
```

Examples:

- Two Sum
- Four Sum (optimized versions)

4. Is the problem asking about subarrays?

Whenever you see:

- Count subarrays
- Longest subarray
- Sum equals k
- Equal number of 0s and 1s

think immediately about

Prefix Sum + Hash Map

Instead of storing every subarray, store previously seen prefix sums.

Examples:

- Subarray Sum Equals K
- Binary Subarrays With Sum
- Contiguous Array

5. Does the problem involve XOR?

Whenever XOR appears in the statement,

$$a \oplus b = c$$

replace Prefix Sum with Prefix XOR.

Think:

Prefix XOR + Hash Map

Examples:

- Count XOR Subarrays
- Subarray XOR Equals K

6. Does the window need unique elements?

Keywords such as

- Longest substring
- No repeating characters
- Distinct elements

almost always indicate

Sliding Window + Hash Map

Examples:

- Longest Substring Without Repeating Characters
- Minimum Window Substring
- Find All Anagrams

Golden Rule

Do not memorize solutions.
Instead, ask yourself:

“What information do I need to remember while traversing the array?”

If the answer is:

- Frequency → `unordered_map`
- Presence → `unordered_set`
- Previous Index → `unordered_map`
- Prefix Sum → Hash Map
- Prefix XOR → Hash Map
- Character Counts → Sliding Window + Hash Map

then the problem is very likely a hashing problem.

2 STL

2.1 `unordered_map`

Used for frequencies, prefix sums, and lookups.

```
1 mp[x]++;  
2 if(mp.count(x)){  
3     mp.erase(x);
```

2.2 `unordered_set`

Used when only existence matters.

```
1 st.insert(x);  
2 st.count(x);
```

3 Complexities

Container	Search	Insert	Delete
unordered_map	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
unordered_set	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
map	$O(\log n)$	$O(\log n)$	$O(\log n)$
set	$O(\log n)$	$O(\log n)$	$O(\log n)$

4 Core Patterns

4.1 Frequency Counting

Questions:

- Majority Element
- Top K Frequent Elements
- Valid Anagram
- Group Anagrams

Template:

```
1 for(int x: nums) freq[x]++;
```

4.2 Lookup Pattern

Typical problem: Two Sum.

```
1 for(int i=0;i<nums.size();i++){
2     if(mp.count(target-nums[i]))
3         return {mp[target-nums[i]],i};
4     mp[nums[i]]=i;
5 }
```

4.3 Prefix Sum + Hash Map

Important problems:

- Subarray Sum Equals K
- Binary Subarrays With Sum
- Contiguous Array

Template:

```
1 unordered_map<int,int> mp;
2 mp[0]=1;
3 int sum=0,ans=0;
4 for(int x:nums){
5     sum+=x;
6     ans+=mp[sum-k];
7     mp[sum]++;
8 }
```

4.4 Prefix XOR

Use XOR instead of sum.

```
1 mp[0]=1;
```

4.5 Sliding Window + Hashing

Problems:

- Longest Substring Without Repeating Characters
- Minimum Window Substring
- Find All Anagrams

5 Recognition Table

Question Clue	Pattern
Count occurrences	Frequency map
Pair equals target	Lookup map
Count subarrays	Prefix sum + map
XOR subarrays	Prefix XOR
Longest consecutive	Hash set
Distinct window	Sliding window + map
Modulo condition	Prefix modulo

6 Common Mistakes

- Forgetting `mp[0]=1` for prefix sum counting.
- Overwriting first occurrence when longest subarray is required.
- Using `map` instead of `unordered_map` unnecessarily.

7 Important Interview Problems

1. Two Sum
2. Contains Duplicate
3. Majority Element
4. Valid Anagram
5. Group Anagrams
6. Longest Consecutive Sequence
7. Subarray Sum Equals K
8. Binary Subarrays With Sum
9. Count Nice Subarrays

10. Contiguous Array
11. Continuous Subarray Sum
12. Subarray XOR Equals K
13. Longest Substring Without Repeating Characters
14. Minimum Window Substring
15. Find All Anagrams in a String